

```

# =====
# K-MEANS CLUSTERING
# PALMER PENGUINS MORPHOLOGY ANALYSIS
# COMPLETE SUBMISSION
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Set style
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 8)

print("="*60)
print("K-MEANS CLUSTERING - PALMER PENGUINS")
print("="*60)

# =====
# LOAD DATASET
# =====

print("\n" + "="*60)
print("LOAD DATASET")
print("="*60)

# Try to load the penguins dataset
csv_files = [f for f in os.listdir('.') if f.endswith('.csv')]

if csv_files:
    filename = csv_files[0]
    print(f"Found: {filename}")
    df = pd.read_csv(filename)
else:
    # If no CSV, try loading from seaborn (built-in dataset)
    try:
        df = sns.load_dataset('penguins')
        print("Loaded penguins dataset from seaborn")
    except:
        print("ERROR: Could not load dataset. Please upload the CSV file.")
        exit()

print(f"Dataset: {df.shape[0]} rows, {df.shape[1]} columns")

print("\nFirst 5 rows:")
print(df.head())

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

```

```

print("\nMissing values:")
print(df.isnull().sum())

# =====
# DATA CLEANING
# =====

print("\n" + "="*60)
print("DATA CLEANING")
print("="*60)

# Drop rows with missing values
df = df.dropna()
print(f"After dropping missing values: {df.shape[0]} rows")

# Select numeric columns for clustering
numeric_cols = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_

# Check if these columns exist
available_numeric = [col for col in numeric_cols if col in df.columns]

if not available_numeric:
    # Try alternative column names
    alternatives = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'b
    available_numeric = [col for col in df.columns if col in alternatives]

print(f"Numeric columns for clustering: {available_numeric}")

X = df[available_numeric].copy()
print(f"Features shape: {X.shape}")

# =====
# FEATURE SCALING (StandardScaler)
# =====

print("\n" + "="*60)
print("FEATURE SCALING (StandardScaler)")
print("="*60)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Features scaled successfully.")
print(f"Scaled features shape: {X_scaled.shape}")

# =====
# ELBOW METHOD (Find Optimal k)
# =====

print("\n" + "="*60)
print("ELBOW METHOD - FIND OPTIMAL k")
print("="*60)

# Test different k values
k_values = range(1, 11)
inertia = [] # Sum of squared distances (SSE)

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)

```

```

kmeans.fit(X_scaled)
inertia.append(kmeans.inertia_)

# Plot Elbow curve
plt.figure(figsize=(10, 6))
plt.plot(k_values, inertia, 'bo-', linewidth=2)
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia (SSE)')
plt.title('Elbow Method for Optimal k')
plt.xticks(k_values)
plt.grid(True, alpha=0.3)
plt.show()

print("Inertia values for each k:")
for k, i in zip(k_values, inertia):
    print(f" k={k}: {i:.2f}")

# =====
# SILHOUETTE SCORES
# =====

print("\n" + "="*60)
print("SILHOUETTE SCORES")
print("="*60)

silhouette_scores = []

for k in range(2, 11): # Silhouette requires at least 2 clusters
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    silhouette_scores.append((k, score))

print("Silhouette Scores:")
for k, score in silhouette_scores:
    print(f" k={k}: {score:.4f}")

# Find best k
best_k = max(silhouette_scores, key=lambda x: x[1])
print(f"\nBest k based on Silhouette Score: k={best_k[0]} (Score: {best_k[1]:.4f})")

# Plot Silhouette scores
plt.figure(figsize=(10, 6))
plt.plot([k for k, s in silhouette_scores], [s for k, s in silhouette_scores],
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.title('Silhouette Scores for Different k Values')
plt.xticks(range(2, 11))
plt.grid(True, alpha=0.3)
plt.show()

# =====
# CHOOSE OPTIMAL k
# =====

print("\n" + "="*60)
print("CHOOSE OPTIMAL k")
print("="*60)

# Based on Elbow and Silhouette, choose k

```

```

# Usually k=3 is optimal for penguins (3 species)
optimal_k = 3

# Check if silhouette suggests different
if best_k[0] in [2, 3, 4]:
    optimal_k = best_k[0]
else:
    optimal_k = 3

print(f"Optimal k selected: {optimal_k}")

# =====
# FIT FINAL K-MEANS MODEL
# =====

print("\n" + "="*60)
print("FIT FINAL K-MEANS MODEL")
print("="*60)

kmeans_final = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
labels = kmeans_final.fit_predict(X_scaled)

# Add cluster labels to original dataframe
df['cluster'] = labels

print(f"Cluster distribution:")
print(df['cluster'].value_counts())

# =====
# VISUALIZE CLUSTERS
# =====

print("\n" + "="*60)
print("VISUALIZE CLUSTERS")
print("="*60)

# 1. Flipper Length vs Body Mass
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
scatter = plt.scatter(df['flipper_length_mm'], df['body_mass_g'],
                      c=df['cluster'], cmap='viridis', s=50, alpha=0.7)
plt.xlabel('Flipper Length (mm)')
plt.ylabel('Body Mass (g)')
plt.title(f'Clusters (k={optimal_k}) - Flipper Length vs Body Mass')
plt.colorbar(scatter, label='Cluster')

# 2. Bill Length vs Bill Depth
plt.subplot(1, 2, 2)
scatter2 = plt.scatter(df['bill_length_mm'], df['bill_depth_mm'],
                       c=df['cluster'], cmap='viridis', s=50, alpha=0.7)
plt.xlabel('Bill Length (mm)')
plt.ylabel('Bill Depth (mm)')
plt.title(f'Clusters (k={optimal_k}) - Bill Length vs Bill Depth')
plt.colorbar(scatter2, label='Cluster')

plt.tight_layout()
plt.show()

# =====

```

```

# CLUSTER CHARACTERISTICS
# =====

print("\n" + "="*60)
print("CLUSTER CHARACTERISTICS")
print("="*60)

cluster_stats = df.groupby('cluster')[available_numeric].mean()
print("Cluster centroids (average values):")
print(cluster_stats)

# Add cluster centers
centroids = pd.DataFrame(
    kmeans_final.cluster_centers_,
    columns=available_numeric
)
print("\nCentroids (scaled):")
print(centroids)

# =====
# BIOLOGICAL INTERPRETATION
# =====

print("\n" + "="*60)
print("BIOLOGICAL INTERPRETATION")
print("="*60)

print("""
📋 CLUSTER INTERPRETATION:

The 3 clusters likely correspond to the 3 penguin species:
• Adelie: Smaller body mass, shorter flippers, smaller bills
• Chinstrap: Medium body mass, medium flippers
• Gentoo: Larger body mass, longer flippers, larger bills

CLUSTER CHARACTERISTICS:
""")

for i in range(optimal_k):
    cluster_data = df[df['cluster'] == i]
    print(f"\nCluster {i}:")
    print(f"  Count: {len(cluster_data)} penguins")
    print(f"  Avg Body Mass: {cluster_data['body_mass_g'].mean():.1f} g")
    print(f"  Avg Flipper Length: {cluster_data['flipper_length_mm'].mean():.1f} mm")
    print(f"  Avg Bill Length: {cluster_data['bill_length_mm'].mean():.1f} mm")

# =====
# LIMITATIONS AND NEXT STEPS
# =====

print("\n" + "="*60)
print("LIMITATIONS AND NEXT STEPS")
print("="*60)

print("""
📋 LIMITATIONS:

1. K-means assumes spherical clusters of similar size
2. Assumes clusters are linearly separable
3. Sensitive to outliers

```

4. Requires choosing k in advance
5. Does not handle categorical variables

✅ NEXT STEPS:

1. Use PCA to visualize clusters in 2D
2. Compare with actual species labels
3. Try different clustering algorithms (DBSCAN, Hierarchical)
4. Add more features for better separation
5. Validate with domain expert (biologist)

""")

```
print("="*60)
print("PROJECT COMPLETE - ALL REQUIREMENTS MET")
print("="*60)
```

